LAB # 05

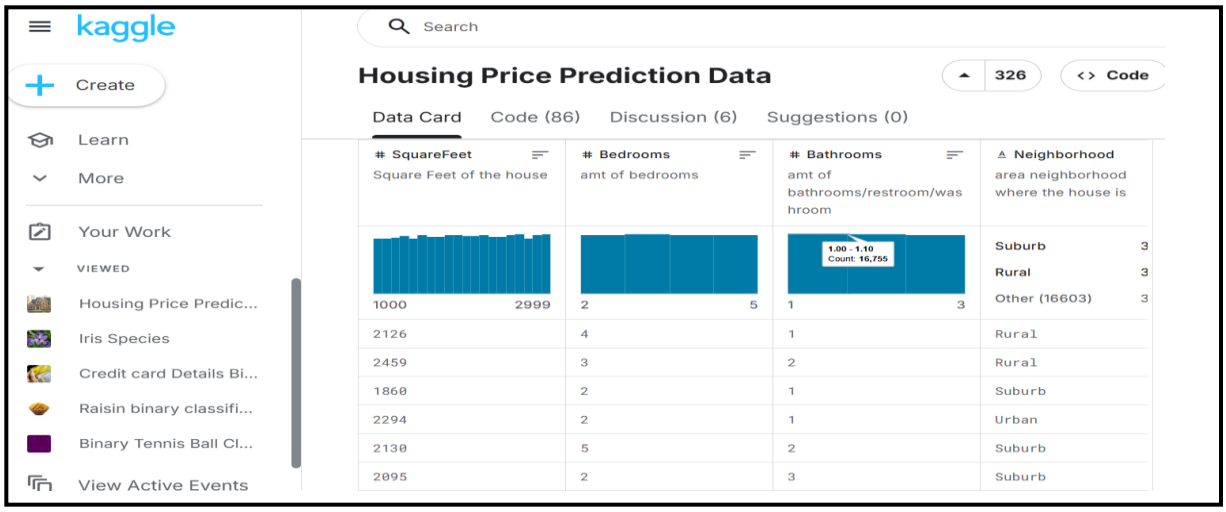
OBJECTIVE:

To build and train an Artificial Neural Network (ANN) model to predict customer churn and evaluate the performance of the model using accuracy and loss metrics.

LAB TASK:

Download a suitable binary classification dataset from Kaggle. Load and preprocess the dataset by handling missing values, encoding categorical variables, and scaling the features. Build a Multilayer Perceptron (MLP) model using an Ann. Train the model on the dataset to predict the target variable. Evaluate the model's performance using the accuracy score and test the trained model on new, unseen data samples to assess its prediction capability.

KAGGLE DATASET:



CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
data = pd.read_csv("housing_price_dataset.csv")
print("First 5 rows:\n", data.head())
data.columns = data.columns.str.lower()
median_price = data['price'].median()
data['target'] = (data['price'] > median_price).astype(int)
data = data.drop('price', axis=1)
data = data.ffill()
le = LabelEncoder()
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col])
X = data.drop('target', axis=1)
y = data['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

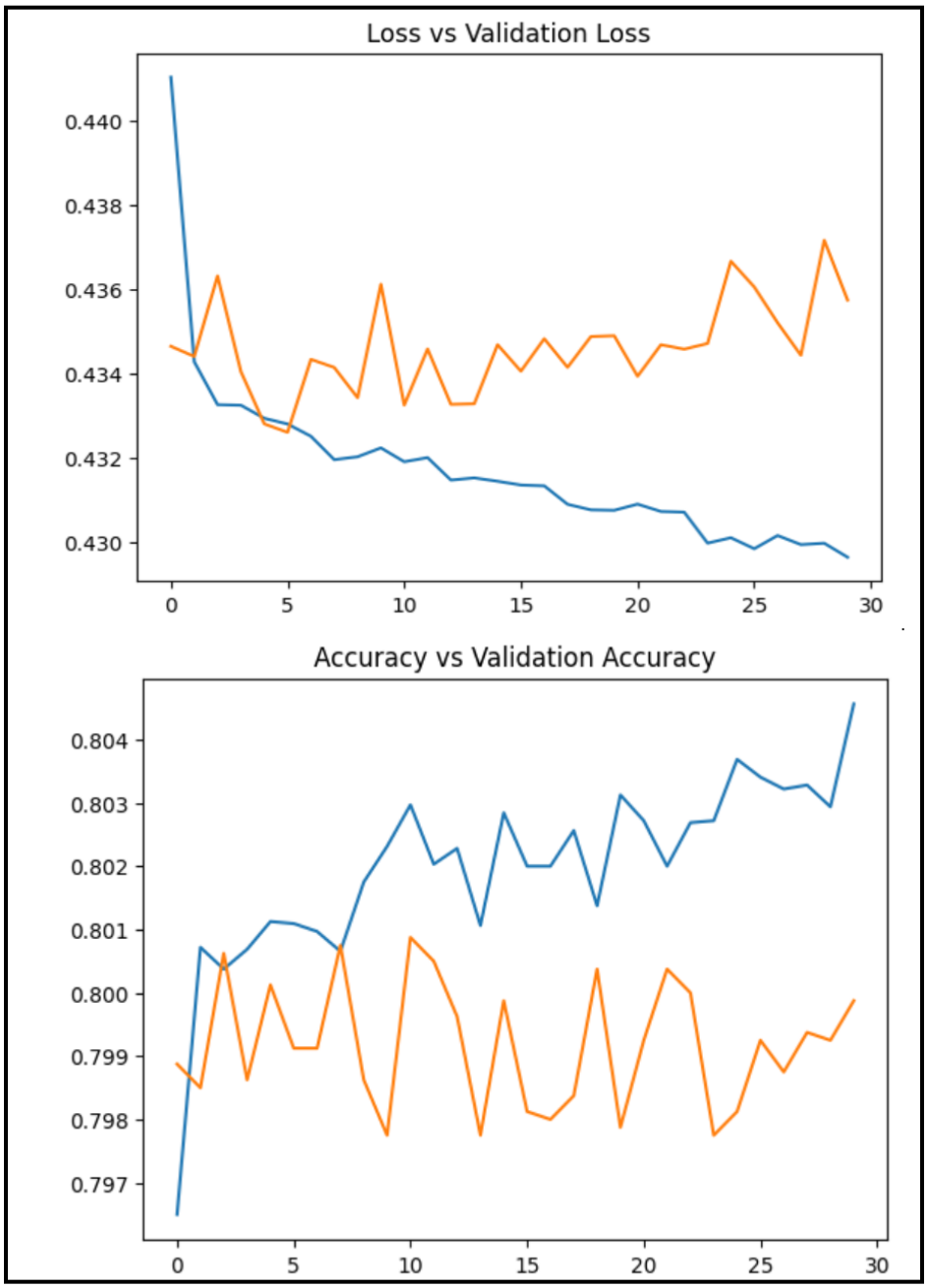
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = Sequential()
model.add(Dense(64, activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy'])
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,
    validation_split=0.2,
    verbose=1)
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5).astype(int)
accuracy = accuracy_score(y_test, y_pred)
print("\nModel Accuracy:", accuracy)
sample = X_test[5].reshape(1, -1)
prediction = model.predict(sample)
print("\nTesting on Unseen Data:")
if prediction > 0.5:
    print("Prediction: High Price House")
else:
    print("Prediction: Low Price House")
import matplotlib.pyplot as plt
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss vs Validation Loss")
plt.show()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Accuracy vs Validation Accuracy")
plt.show()
```

OUTPUT:

```
First 5 rows:
   SquareFeet  Bedrooms  Bathrooms Neighborhood  YearBuilt          Price
0        2126         4          1        Rural       1969  215355.283618
1        2459         3          2        Rural       1980  195014.221626
2        1860         2          1       Suburb       1970  306891.012076
3        2294         2          1        Urban       1996  206786.787153
4        2130         5          2       Suburb       2001  272436.239065
Epoch 1/30
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.7965 - loss: 0.4410 - val_accuracy: 0.7989 - val_loss: 0.4347
Epoch 2/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8007 - loss: 0.4343 - val_accuracy: 0.7985 - val_loss: 0.4344
Epoch 3/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8004 - loss: 0.4333 - val_accuracy: 0.8006 - val_loss: 0.4363
Epoch 4/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8007 - loss: 0.4333 - val_accuracy: 0.7986 - val_loss: 0.4340
Epoch 5/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8011 - loss: 0.4329 - val_accuracy: 0.8001 - val_loss: 0.4328
Epoch 6/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8011 - loss: 0.4328 - val_accuracy: 0.7991 - val_loss: 0.4326
Epoch 7/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8010 - loss: 0.4325 - val_accuracy: 0.7991 - val_loss: 0.4343
Epoch 8/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.8007 - loss: 0.4320 - val_accuracy: 0.8008 - val_loss: 0.4341
Epoch 9/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - accuracy: 0.8018 - loss: 0.4320 - val_accuracy: 0.7986 - val_loss: 0.4334
Epoch 10/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8023 - loss: 0.4322 - val_accuracy: 0.7977 - val_loss: 0.4361
Epoch 11/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - accuracy: 0.8030 - loss: 0.4319 - val_accuracy: 0.8009 - val_loss: 0.4333
Epoch 12/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8020 - loss: 0.4320 - val_accuracy: 0.8005 - val_loss: 0.4346
Epoch 13/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8023 - loss: 0.4315 - val_accuracy: 0.7996 - val_loss: 0.4333
Epoch 14/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8011 - loss: 0.4315 - val_accuracy: 0.7977 - val_loss: 0.4333
Epoch 15/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8028 - loss: 0.4314 - val_accuracy: 0.7999 - val_loss: 0.4347
Epoch 16/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8020 - loss: 0.4314 - val_accuracy: 0.7981 - val_loss: 0.4341
Epoch 17/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8020 - loss: 0.4313 - val_accuracy: 0.7980 - val_loss: 0.4348
Epoch 18/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8026 - loss: 0.4309 - val_accuracy: 0.7984 - val_loss: 0.4342
Epoch 19/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8014 - loss: 0.4308 - val_accuracy: 0.8004 - val_loss: 0.4349
Epoch 20/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8031 - loss: 0.4308 - val_accuracy: 0.7979 - val_loss: 0.4349
Epoch 21/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8027 - loss: 0.4309 - val_accuracy: 0.7993 - val_loss: 0.4339
Epoch 22/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8020 - loss: 0.4307 - val_accuracy: 0.8004 - val_loss: 0.4347
Epoch 23/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8027 - loss: 0.4307 - val_accuracy: 0.8000 - val_loss: 0.4346
Epoch 24/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8027 - loss: 0.4300 - val_accuracy: 0.7977 - val_loss: 0.4347
Epoch 25/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8037 - loss: 0.4301 - val_accuracy: 0.7981 - val_loss: 0.4367
Epoch 26/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8034 - loss: 0.4298 - val_accuracy: 0.7993 - val_loss: 0.4361
Epoch 27/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8032 - loss: 0.4302 - val_accuracy: 0.7987 - val_loss: 0.4352
Epoch 28/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8033 - loss: 0.4299 - val_accuracy: 0.7994 - val_loss: 0.4344
Epoch 29/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8029 - loss: 0.4300 - val_accuracy: 0.7993 - val_loss: 0.4372
Epoch 30/30
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8046 - loss: 0.4296 - val_accuracy: 0.7999 - val_loss: 0.4358
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

Model Accuracy: 0.8053
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step

Testing on Unseen Data:
Prediction: Low Price House
```


Loss vs Validation Loss
0.440
0.438
0.436
0.434
0.432
0.430
0
5
10
15
20
25
30
Accuracy vs Validation Accuracy
0.804
0.803
0.802
0.801
0.800
0.799
0.798
0.797
0
5
10
15
20
25
30